A preregistered frame-only census of game mechanics, fitted on ARC-AGI-3, failed its validity test on MiniGrid/BabyAI and Procgen

Rahul Kumar Mishra

Independent Researcher, Ranchi, Jharkhand, India

ORCID 0009-0004-9671-4055

mishra.rahul98@gmail.com

Article type: Empirical Article

This is the preprint version of an article submitted to the Journal of Trial and Error; it is published under the CC-BY 4.0 license.

Keywords

preregistration; negative result; game-playing agents; benchmark mechanics; MiniGrid, Procgen, ARC-AGI-3

Take-home message

A probe battery fitted on one game benchmark did not measure the game mechanics it was built to measure on two other benchmarks. The preregistered validity test caught this before any claim was made. Test an instrument against known truth where its parameters are set, before running it elsewhere.

Abstract

Before any confirmatory run, I registered the claim that a fixed battery of frame-only probes, the assumption census, would report five mechanical assumptions of unseen games as each game's source code states them. It would also predict on which games assumption-driven frame-only agents complete the first level. I froze every census parameter on eight ARC-AGI-3 games. Test V1 compared the census with source-derived truth tables; test V2 compared a census score with first-level completion by three agents. The census and the agents ran on 161 MiniGrid/BabyAI environments under two action conditions and on 14 Procgen games under two camera settings, with 10 seeds and 20,000 actions per run,

10,500 runs in all. By the registered outcome table the result is a refutation, under the clause “V1 fails on any suite”. V1 failed for three of five assumptions under absolute moves and for four of five under primitive actions on MiniGrid/BabyAI, and for four of five on Procgen. No agent passed V2 on any suite. On all six primary MiniGrid/BabyAI lines, a nuisance baseline built from grid area, action count and color count ranked the games better than the census did. An exploratory reading made after the outcome suggests that for four of the five assumptions the probes and the truth definitions were not the same construct. The lesson is procedural: check an instrument against known truth where its parameters are set, and on the pilot games, before any confirmatory use.

Introduction

A frame-only game agent sees the picture on the screen and is told when a level or the game ends, and nothing more. It receives no score, no list of objects, no text and no access to the game’s code, and it chooses an action from a fixed set after each frame. Agents of this kind are a natural baseline for benchmarks of unseen games, such as ARC-AGI-3, where each game must be learned from interaction alone. The agents studied here carry built-in assumptions about how games work, for example that one object on the screen is the player and that the arrow keys move it one cell at a time. The idea this study tested is that such an agent does well where its assumptions hold and poorly where they fail. If one could tell, before running any agent, which assumptions an unseen game satisfies, one could then predict where such agents will finish the first level, and choose or build agents accordingly.

The assumption census was meant to supply that prediction. It is a fixed set of probes that reads frames only and reports, for each game, whether five mechanical assumptions hold. A1, input coupling, says that the actions change the picture and that different actions change it differently. A2, one controlled object, says that exactly one object on the screen moves as a fixed function of the action. A3, lattice motion, says that each movement action shifts that object by a constant step whenever it is not blocked. A4, blocking by a fixed obstacle set, says that whenever a move has no effect, a fixed set of obstacles explains it, and that this set changes only through the agent’s own actions. A5, place goal, says that a level ends when the controlled object enters a particular place, with no other condition at that moment. Each assumption is a property of the game, and each can be read off the game’s source code. The census tries to infer them from play.

I preregistered the claim because it is easy to fool oneself with an instrument of this kind. I had tuned its parameters while developing a planner on a small set of games, and almost any reading of its output on new games can be made to look sensible after the fact. A preregistration fixes, before any confirmatory data exist, the claim, the games, the truth against which the instrument is scored, the tests, their thresholds and the consequence of each outcome. The author then commits to reporting what the tests return, whatever it is, and to changing the protocol only through dated amendments that state what changed and

why. Bertinetto et al. (2021) describe this model for machine learning research, where the experimental protocol is fixed and reviewed before the experiments are run.

The closest earlier attempt to predict agent success from game mechanics reported a mainly negative result. Bontrager et al. (2016) read mechanics features from the rule files of general video game playing games. A clustering of the games by those features agreed poorly with a clustering by controller performance, and the authors concluded that the features were not good indicators of performance. A selection agent built on two of the features did beat each of its constituent controllers on the same games. One possible objection to that design is that features read from rules say nothing about whether an agent could perceive them. Another is that the target, the best controller from a portfolio, was not tied to any assumption the features express. This study was designed to meet both objections. The census measures its features from frames, as an agent would. Its accuracy against source-derived truth is tested separately, as test V1. Its prediction target, first-level completion by a stated class of agents that rely on the assumptions, is tested as V2, registered before any run.

The census builds on earlier work on finding, from frames alone, the part of the screen that the actions control. Bellemare et al. (2012) brought contingency awareness into reinforcement learning, defining contingent regions as the parts of an observation whose next value depends on the agent's action. Choi et al. (2019) learned controllable regions with a neural network. The first two census assumptions take that idea from them and replace the learned components with fixed counting rules.

I developed the census on eight of the 25 public ARC-AGI-3 games and kept the other 17 as a held-out set whose source code I never opened. The split is fixed in a file of the repository. That file held 18 games in the held-out set until 21 September 2026, when I moved one of them, dc22, to the development set to debug the agent's wall learning on it, before any detailed study of the game. The reason for a fixed split is the problem of holdout reuse: when the same held-out set is consulted again and again while choices are made, choices can come to fit it (Blum & Hardt, 2015). Roelofs et al. (2019) measured this effect across many machine learning competitions and found little of it. The two further suites, MiniGrid/BabyAI, which joins MiniGrid (Chevalier-Boisvert et al., 2023) and BabyAI (Chevalier-Boisvert et al., 2019), and Procgen (Cobbe et al., 2020), were chosen to test whether the census transfers to games it was never tuned on.

The outcome, by the registered outcome table, is a refutation: the census failed the validity test V1 on both confirmatory suites, and no agent passed the prediction test V2 on any suite.

Method

The instrument

The census reads the run of one agent, the planner N described below, on a game for 20,000 actions per seed. It uses the frames of that run and the signals that a level or the game has ended, and nothing else: no reward, no symbolic state, no text. From the planner's report at the end of the run it makes five decisions per seed, with rules written into the registered protocol and implemented in the frozen analysis code. A1 holds when some key has a learned shift and the tried keys' shifts are not all equal. A2 holds when some key was accepted as moving one object and the planner located that object. Acceptance needs three agreeing presses with a margin of 2:1 over the next most frequent displacement.

A3 holds when A2 holds and the planner's model of the first level did not end in a failure of prediction or localization. A4 holds when A3 holds and the planner never walked through a color it had classified as a wall. A5 holds when a place recorded at a level end later ended a level again. It fails when such a place was reached without a level end and never confirmed, and it is undetermined otherwise. The verdict for a game is the majority over its 10 seeds: it holds if at least half of the seeds say so, else it fails if at least half say so, else it is undetermined.

I froze every parameter of the census before registration and copied each into the protocol from its source file. They include three agreeing presses to accept a key, the 2:1 margin, at most eight presses per key while learning, a planner search limit of 40000 nodes, and a motion detector that needs at least 2 changed cells. Six files hold the census and its two agents. Their git blob hashes are registered (Appendix D), and the runner refused to start if any of them differed.

I set these parameters while developing the planner on the eight ARC-AGI-3 training games. The protocol provided for a truth table derived from source for those eight games, and for V1 on them, labeled development, once that table existed. I never wrote the table, so the census was never scored against source-derived truth on its development instance. The registered ARC-AGI-3 baseline ran my agents R and N on all 25 public games with 10 seeds and 20,000 actions under the registered code, 500 runs in all. The protocol also listed I1 for ARC-AGI-3, and I1 was not run there under the registered code. Both omissions are among the deviations below.

The suites and how each is presented

The frozen code presses six key identifiers only, so every suite is presented to it through a fixed key map of at most six actions. On MiniGrid/BabyAI, the environments of MiniGrid (Chevalier-Boisvert et al., 2023) and BabyAI (Chevalier-Boisvert et al., 2019) run with minigrid 3.1.0, gymnasium 1.3.0 and Python 3.11.9, and there are two conditions. Under primitive actions the keys are turn left, turn right, forward, pickup, drop and toggle. Under

absolute moves they are north, south, west and east, each a turn to face that direction followed by a step forward, plus pickup and toggle. The action “done” is offered in neither condition, and “drop” is not offered under absolute moves. The grid is rendered with 8 × 8 pixel tiles, each distinct tile pattern receives an integer index, and the four orientations of the agent’s triangle are merged into one index. The grid is then placed at the top left of a 64 × 64 canvas, one row down, because the frozen code reads rows 1 to 62 only.

On Procgen (procgen 0.10.7, Python 3.10, easy mode, sequential levels) the keys are up, down, left, right and two special actions. Frames are 64 × 64 color images, mapped to integer indices by exact color and capped at 64 indices. The frozen code does not see the top and bottom pixel rows, which was recorded and not corrected. The primary condition draws the whole level into the frame (center_agent off). In that view the agent can be very small, and the census detects motion only when at least 2 cells change. The protocol therefore registered an operating range before any run: the games whose agent spans at least 2 pixels. Nine of the 14 confirmatory games fall in that range, and Procgen results are reported over all 14 games and again over these nine.

The secondary condition, a view centered on the agent, turned out to be identical to the primary one on most games. Ten Procgen games set the camera option themselves at every level start, and nine of them are confirmatory games, exactly the nine in the operating range. A camera check found 401 of 401 frames identical between the two conditions on those ten games, and a difference from the first frame on the other six. The secondary condition is therefore distinct on five games only, and it is reported there and never tested.

Truth from source

Each suite has a truth table, committed before the protocol was registered, with one line per environment. For MiniGrid/BabyAI I read the entries by hand from the installed minigrid 3.1.0 source, once per environment class and its step function or instruction verifier. For Procgen I read them from the C++ source at tag 0.10.7. Each entry names the file and line that justify it, and the definitions of A1 to A5 used by the truth tables are written out in the protocol. Under primitive actions A3 fails on every MiniGrid/BabyAI environment, because the displacement of forward depends on which way the agent faces. On the 161 confirmatory MiniGrid/BabyAI environments, A1, A2 and A3 under absolute moves hold everywhere, A4 fails on 5 and A5 holds on 36. On the 14 confirmatory Procgen games, A2 holds on 6, A3 on 2, A4 on 13 and A5 on 6. The truth tables are the registered answer key of test V1; the census is the instrument under test.

Agents

Three agents ran on every game, seed and condition. R is my explorer, which tries actions and records the frames it has seen, and it is evidence for V2. N is my planner, which learns a step vector for each key, locates the controlled object, plans toward unexplored places and tests goal hypotheses. N uses the five assumptions by construction, so its V2 lines are

an instrument check only. Its runs also supply the census. I1 is the graph-based explorer of Rudakov et al. (2025), run unchanged behind a boundary adapter, and it is evidence for V2. I1's own code maps five actions, so it has no toggle on MiniGrid/BabyAI and no second special action on Procgen.

The preregistration

I registered version v1 of the protocol on 27 September 2026 as git tag prereg-v1 on commit 19f2a9f25043997d1deffd5976f2f2150e5f086f. On the same day I registered it on the Open Science Framework (OSF) as an open-ended registration (https://osf.io/2znhw). It fixed the claim, the agents, the census rules and constants, the truth tables, the suites and their splits between pilot and confirmatory games, the seeds and budget, the tests and thresholds, the analysis code and the outcome table. The protocol gives each census constant with its file, line and commit. LEARN_AGREE = 3 is at scripts/g_nav.py:55, the 2:1 margin at scripts/g_nav.py:756, LEARN_MAX = 8 at scripts/g_nav.py:56 and the search limit of 40000 nodes at scripts/g_nav.py:54, all at commit ccc4d57. The motion threshold of 2 cells is at scripts/g_motion.py:20, commit 378345f.

The claim sentence differs in wording between the two registered texts of v1. The git-tagged protocol names the goal-identification cost G as part of the battery and says "in three suites". The OSF text, quoted in the Purpose section, describes G as "measured and reported alongside, not tested" and names no number of suites. The tests, thresholds and outcome table are the same in both.

The protocol also fixed the pilot rules. The pilot games, six MiniGrid/BabyAI environments and two Procgen games, could be run before the confirmatory set and are excluded from every confirmatory analysis. The pilot could change adapter code but never a census constant, threshold, truth entry or frozen file. Every change had to be logged in the next version's dated diff section. After the pilot I registered version v1.1 on 27 September 2026 as git tag prereg-v1.1 on commit 8b2b2b9b432dbea4ba5f62cfceb1aad5558d1ecc, and on the same day on the OSF (https://osf.io/zu5vq). Both registrations are under embargo until 30 June 2030, and their registered texts are in the repository. I wrote version v1.2 after the outcome and tagged it prereg-v1.2 on commit 66e560645d9d98bca39ee7da9b1039ada1bc579b on 13 October 2026.

The amendment trail has 16 entries, D1 to D16, given in full in Appendix A. D1 to D15 form the v1.1 diff, written on 27 September 2026 after the pilot and before any confirmatory game was played. D1 to D4 fix how each suite is presented through the six-key limit, D5 sets the per-run time caps, D6 the runner, D7 the boundary of I1 and D8 the infrastructure checks. D9 and D9b set the exclusions from V2, and D10 and D11 the analysis code at the tag. D12 sets a time cap for ARC-AGI-3 and the repeat of eight ARC runs that an unregistered clock had cut short. D13 records the Procgen camera identity, D14 the absence of any adapter change after the pilot, and D15 a separate replay planned for G. D16 is the only entry of v1.2, written after the outcome: it withdraws G without measuring it.

No threshold, parameter, exclusion list, truth entry or analysis script changed after the prereg-v1.1 tag.

Tests

Test V1, instrument validity, compares the census's game verdict with the truth table, per suite, condition and assumption, over the confirmatory games. It passes when the share of agreeing games is at least 0.90 and Cohen's κ is at least 0.70. It fails when agreement is below 0.75, and anything between is "neither". κ is computed only where both truth values have at least 3 games; elsewhere agreement decides alone. An undetermined census verdict never counts as agreement, and for κ it is mapped to the value opposite the truth. That treatment of "undetermined" is in the registered text, which reads "An 'undetermined' census never agrees. For κ it is mapped to the value opposite the truth." The frozen analysis code at the tag implements the same rule.

Test V2, predictive validity, works per suite, condition and agent. Each game has a census score, the number of the five assumptions whose verdict holds (undetermined does not hold). It also has a conjunctive prediction, that A2, A3, A4 and A5 all hold, and an outcome: the agent completed the first level in at least half of its 10 seeds. The score is judged by the area under the receiver operating characteristic curve (AUROC) for the outcome. V2 passes when four conditions all hold. The AUROC is at least 0.80. The lower end of its 95% bootstrap interval, from 2,000 resamples of games with seed 20260927, is above 0.60. The balanced accuracy of the conjunctive prediction is at least 0.80. The AUROC exceeds that of a nuisance baseline by at least 0.10.

V2 fails when the AUROC is at most 0.65, and the test is void when the share of games completed lies outside [0.10, 0.90]. The nuisance baseline is a logistic regression of the outcome on grid area, number of actions and number of palette colors, fitted in-sample. It is the registered answer to the question of whether the census adds anything to the size and variety of a game. Evidence comes from R and I1, and a suite fails V2 when every non-void evidence agent fails. On MiniGrid/BabyAI the registered text tests V2 under absolute moves and reports it under primitive actions. On ARC-AGI-3, V2 on the 17 held-out games is reported and not tested.

Test V3, parameter transfer, requires every census constant and frozen file to be identical on every suite. The protocol registered it as a check of the adapter and code fields of every row. The analysis code does not compute that check, and no V3 line exists among its outputs. What the record does show is that the runner compared the six frozen files with their registered hashes before each confirmatory block and refused to start on any difference, and that its logs report all six matching for both blocks. The missing row-level check is among the deviations.

Exclusions from V2

Some MiniGrid/BabyAI environments cannot be completed by an agent whose key map lacks a needed action. The registered rule (D9) excludes an environment from an agent's

primary V2 line when, on every seed, success cannot be reached without an action missing from that agent's map. The action may trigger success, or it may be a precondition of every successful episode. One environment has no success at all and is excluded for every agent. For R and N the rule excludes 6 environments under primitive actions and 49 under absolute moves, leaving 155 and 112. In the pilot, I1, which has no toggle, completed no level on a locked-door environment in any of its 20 runs. D9b therefore applied the same rule to I1's own key map, adding the 57 environments where success needs toggle on every seed. I1's primary lines exclude 63 and 78 environments, leaving 98 and 83. Procgen has no exclusion. Lines with the excluded environments included are reported and not tested. The lists, with identifiers, are in Appendix B.

Seeds, budget and caps

Every agent ran with seeds 1000 to 1009 and a budget of 20,000 actions per game, seed and condition. The wall-clock cap per run was three times the longest pilot run: 3300 s on MiniGrid/BabyAI, where the longest pilot run took 1081.6 s, and 7500 s on Procgen, where it took 2481.5 s. The cap for the ARC-AGI-3 baseline was 3600 s. There were 9,660 confirmatory runs on MiniGrid/BabyAI (161 environments, 2 conditions, 3 agents, 10 seeds) and 840 on Procgen (14 games, 2 conditions, 3 agents, 10 seeds), 10,500 in all. Every run reached its budget, with 0 errors and 0 runs cut by the cap. The MiniGrid/BabyAI block ran on one CPU machine; it started on 28 September 2026 and had finished by 13 October 2026.

Checks before and after the runs

Four infrastructure checks were registered, and none computes a census or a test statistic. A fidelity check played 200 random actions twice per game and condition. It found identical frames in 334 of 334 comparisons on MiniGrid/BabyAI and 32 of 32 on Procgen. A pilot check read the 480 pilot runs, 360 on MiniGrid/BabyAI and 120 on Procgen, and found 0 errors and 0 runs short of the budget. It also projected the running time of every confirmatory configuration and found 0 over its cap, the largest projections being 1517.8 s and 2550.0 s. The camera check is described above. A determinism check compared, on the nine Procgen games where the two camera conditions draw the same frames, every secondary run with its primary run. All 270 of 270 pairs were identical in every field except timing and the condition labels, with 0 missing. The analysis code and the determinism check passed 20 of 20 synthetic tests at the tag before the analysis read any confirmatory row.

The registered outcome table

The protocol fixed three outcomes and the venue that each would allow, before any confirmatory run. Support needed V1 to pass on both suites and V2 to pass for R and I1. Partial needed V1 to pass everywhere with a weaker V2. Refute follows when V1 fails on any suite, or V2 fails on MiniGrid/BabyAI under absolute moves, or V2 is void on every suite.

Its registered venue is “repository record only, at most an arXiv note”. The table is reproduced verbatim in the Results (Table 4).

Deviations from the registered protocol

Eight departures from the registered protocol, or from what it implies, are recorded here. None changes a test, a threshold or the outcome. The first is D16, written on 13 October 2026, after the outcome was known. The registered claim says that the goal-identification cost G is “measured and reported alongside, not tested”, and D16 withdrew G without measuring it, because the instrument it accompanies had failed V1. The second concerns the venue. The registered consequence of a refutation is “repository record only, at most an arXiv note”. On 13 October 2026, with no arXiv endorsement available to me, I recorded that the clause would be read as “at most a preprint” and that no journal submission would follow. After the preprint drafts of 14 October 2026 I reversed the second part and chose to submit this article to a journal that publishes negative results. This changes no analysis and no number.

The third is a defect in the frozen analysis code, found on 13 October 2026 and recorded, not corrected. The code computes the two ARC-AGI-3 lines of V2 over all 25 public games, the eight development games included, instead of over the 17 held-out games. It also prints a per-agent verdict on them that has no registered meaning. The fourth is an omission rather than a departure. The pilot runs contained census verdicts for eight games that have truth-table entries, and a V1 comparison on them before the v1.1 tag would have shown the failure before the confirmatory runs. It was not registered and I did not make it; I recorded it as a lesson on 13 October 2026. The fifth I found on 14 October 2026, while writing this article. The registered text tests V2 on MiniGrid/BabyAI under absolute moves and only reports it under primitive actions, but the frozen code prints verdicts, and a suite line, for the primitive-action lines as well. Those verdicts enter no outcome clause.

The last three I found on 14 October 2026, while reviewing this article against the protocol. The sixth: v1 provided for a truth table of the eight ARC-AGI-3 development games and for V1 on them, labeled development, once the table existed. The record listed the table as still outstanding at the v1.1 tag, and I never wrote it. The seventh: v1 listed I1 among the agents to run on ARC-AGI-3, but the registered ARC baseline ran R and N only. The eighth: V3 was registered as a check of every row’s adapter and code fields, and the analysis code never computed it. The ARC-AGI-3 lines are reported and not tested, and V3 matters only for the outcome Support, so none of these three changes the outcome.

Results

The results in this section are the registered tests, exactly as the frozen analysis code wrote them. Where the code prints a verdict that the registered text does not test, the text says so.

Figure 1 and Table 1 give V1. On MiniGrid/BabyAI under absolute moves, A1 and A2 pass and A3, A4 and A5 fail. Under primitive actions only A1 passes; A2, A3, A4 and A5 fail. On Procgen with the whole-level view, over all 14 games, A1 passes and the other four fail. On the nine in-range games, A1 is “neither”, with agreement 0.8889, and A2 to A5 fail. The camera-on condition is reported and not tested. Cohen’s κ is negative for A5 under absolute moves (−0.4125), under primitive actions (−0.3374) and on Procgen (−0.7843), and for A4 under absolute moves (−0.0435).

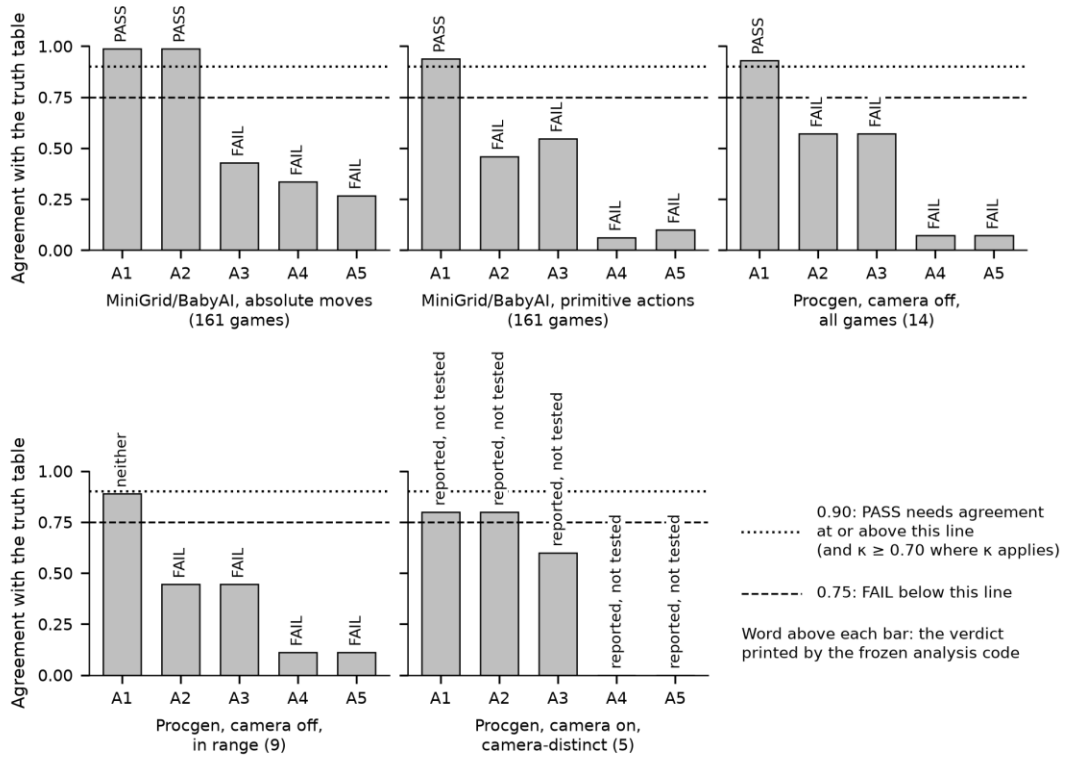


Figure 1. Test V1: agreement between the census and the source-derived truth table, per assumption, for each suite and condition. Dashed line: 0.75, below which V1 fails. Dotted line: 0.90, which V1 needs to pass (with κ of at least 0.70 where κ applies). The word above each bar is the verdict printed by the frozen analysis code; a bar of agreement 0.0000 is not visible. Drawn from docs/tables/v1.json by scripts/manuscript_figures.py. Registered test.

Table 1. Test V1, copied verbatim from docs/tables/v1.md (commit bb0f1a1), as written by the frozen analysis code at prereg-v1.1. Agreement is the share of games where the census verdict equals the truth, and κ is given only where both truth values have at least 3 games. The code’s identifiers are kept. On MiniGrid/BabyAI, “macro” is absolute moves and “raw” primitive actions. On Procgen, “center_off” is the whole-level view and “center_on” the view centered on the agent, “in_range” is the nine games of the operating range, and “camera_distinct” the five games where the two views differ.

suite	condition	subset	assumption	n	truth_holds	agreement	kappa	verdict

suite	condition	subset	assumption	n	truth_holds	agreement	kappa	verdict
minigrid	macro	all	A1	161	161	0.9876		PASS
minigrid	macro	all	A2	161	161	0.9876		PASS
minigrid	macro	all	A3	161	161	0.4286		FAIL
minigrid	macro	all	A4	161	156	0.3354	-0.0435	FAIL
minigrid	macro	all	A5	161	36	0.2671	-0.4125	FAIL
minigrid	raw	all	A1	161	161	0.9379		PASS
minigrid	raw	all	A2	161	161	0.4596		FAIL
minigrid	raw	all	A3	161	0	0.5466		FAIL
minigrid	raw	all	A4	161	156	0.0621	0.0021	FAIL
minigrid	raw	all	A5	161	36	0.0994	-0.3374	FAIL
procgen	center_off	all	A1	14	14	0.9286		PASS
procgen	center_off	all	A2	14	6	0.5714	0.2222	FAIL
procgen	center_off	all	A3	14	2	0.5714		FAIL
procgen	center_off	all	A4	14	13	0.0714		FAIL
procgen	center_off	all	A5	14	6	0.0714	-0.7843	FAIL
procgen	center_off	in_range	A1	9	9	0.8889		neither
procgen	center_off	in_range	A2	9	4	0.4444	0.0000	FAIL
procgen	center_off	in_range	A3	9	2	0.4444		FAIL
procgen	center_off	in_range	A4	9	8	0.1111		FAIL
procgen	center_off	in_range	A5	9	2	0.1111		FAIL
procgen	center_on	camera_distinct	A1	5	5	0.8000		reported, not tested
procgen	center_on	camera_distinct	A2	5	2	0.8000		reported, not tested

suite	condition	subset	assumption	n	truth_holds	agreement	kappa	verdict
procgen	center_on	camera_distinct	A3	5	0	0.6000		reported, not tested
procgen	center_on	camera_distinct	A4	5	5	0.0000		reported, not tested
procgen	center_on	camera_distinct	A5	5	4	0.0000		reported, not tested

Figure 2 and Table 2 give V2. No agent reached PASS on any suite or subset. On MiniGrid/BabyAI under absolute moves, the primary line of R is void, because R completed the first level on 101 of 112 games, a share of 0.9018, above the registered 0.90. I1 is “neither” there, with AUROC 0.7187, and so is N, the instrument check. On the primitive-action lines, which the registered text only reports, the frozen code prints FAIL for R and I1, with AUROC 0.5224 and 0.5370. On Procgen over all 14 games I1 fails, with AUROC 0.6458, and R is “neither”, with 0.7222. On the nine in-range games every agent is “neither”. The intervals on Procgen are wide: over 14 games, let alone nine, the lower end of the 95% interval does not reach 0.60 for any agent, which the protocol anticipated before any run. The interval is not the only obstacle, though. Every Procgen line also has a balanced accuracy of 0.5000, below the registered 0.80, so no Procgen line misses a pass on the interval alone.

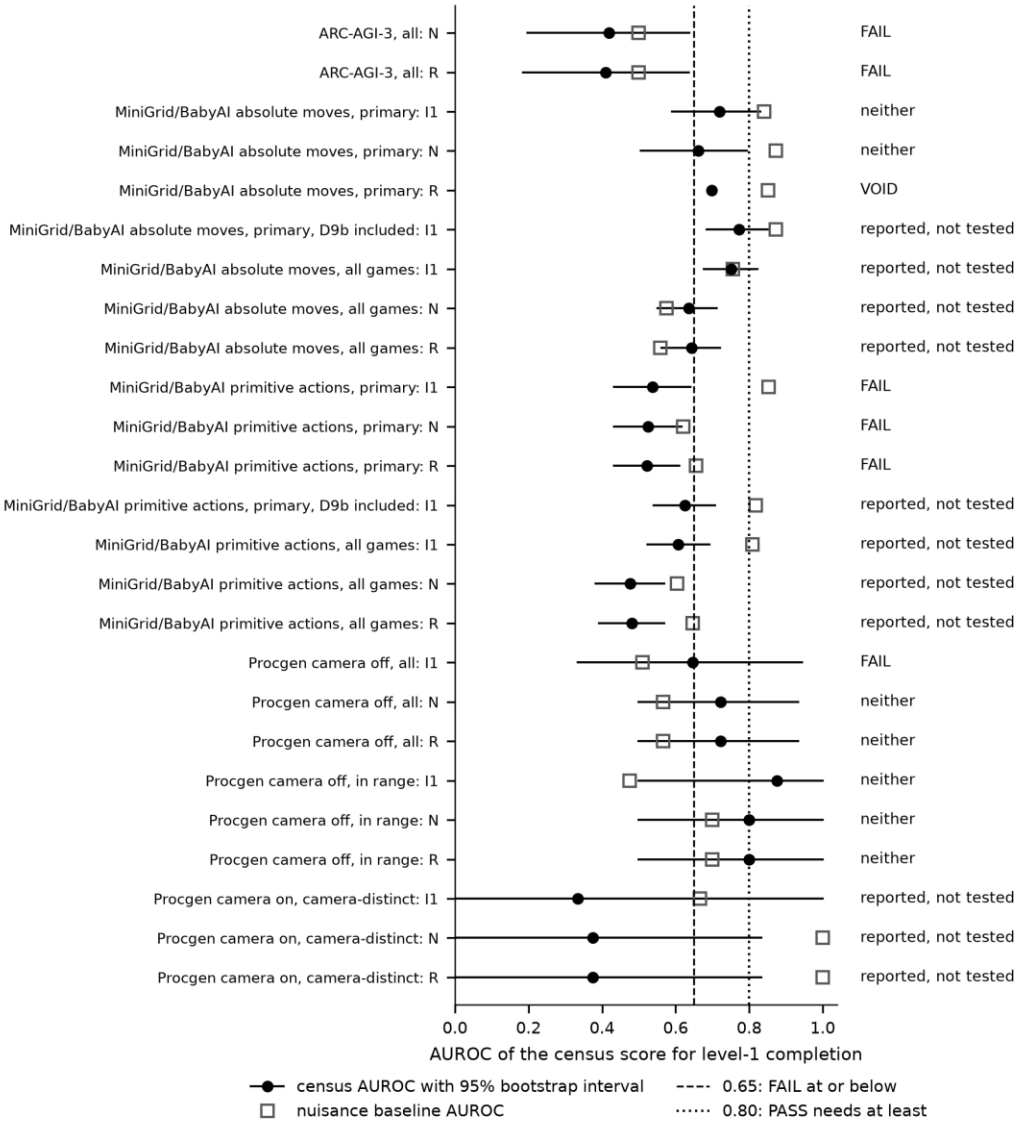


Figure 2. Test V2: AUROC of the census score for first-level completion (filled circle) with its 95% bootstrap interval, and the AUROC of the nuisance baseline (open square), per agent and line. Dashed line: 0.65, at or below which V2 fails. Dotted line: 0.80, which V2 needs to pass, among three further conditions. The word at the right is the verdict printed by the frozen analysis code; the void line has no interval. The two ARC-AGI-3 lines are reported and not tested, and the "FAIL" printed on them has no registered meaning (see Deviations). The primitive-action lines are reported by the registered text, so the "FAIL" printed on their primary lines enters no outcome clause. Drawn from docs/tables/v2.json by scripts/manuscript_figures.py. Registered test.

Table 2. Test V2, copied verbatim from docs/tables/v2.md (commit bb0f1a1), as written by the frozen analysis code at prereg-v1.1, in two parts with the same rows so that it fits the page: Table 2a gives each line and its verdict, Table 2b its statistics. The agents are R, my explorer, N, my planner (instrument check), and I1, the explorer of Rudakov et al. (2025).

Lines marked “primary” exclude the environments of Appendix B, “primary, D9b included” puts back the exclusions specific to I1, and “all games” includes every environment. auroc_B is the AUROC of the nuisance baseline, ci_low and ci_high the 95% bootstrap interval, and balanced_acc the balanced accuracy of the conjunctive prediction. Condition identifiers are as in Table 1; “native” is ARC-AGI-3.

Table 2a. The V2 lines and their verdicts.

suite	condition	subset	agent	role	n	positives	verdict
arc	native	all	N	reported	25	14	FAIL
arc	native	all	R	reported	25	15	FAIL
minigrid	macro	primary	I1	evidence	83	61	neither
minigrid	macro	primary	N	instrument check	112	100	neither
minigrid	macro	primary	R	evidence	112	101	VOID
minigrid	macro	primary, D9b included	I1	evidence	112	61	reported, not tested
minigrid	macro	all games	I1	evidence	161	61	reported, not tested
minigrid	macro	all games	N	instrument check	161	100	reported, not tested
minigrid	macro	all games	R	evidence	161	101	reported, not tested
minigrid	raw	primary	I1	evidence	98	71	FAIL
minigrid	raw	primary	N	instrument check	155	130	FAIL
minigrid	raw	primary	R	evidence	155	126	FAIL
minigrid	raw	primary, D9b included	I1	evidence	155	71	reported, not tested
minigrid	raw	all games	I1	evidence	161	71	reported, not tested
minigrid	raw	all games	N	instrument check	161	130	reported, not tested

suite	condition	subset	agent	role	n	positives	verdict
minigrid	raw	all games	R	evidence	161	126	reported, not tested
procgen	center_off	all	I1	evidence	14	6	FAIL
procgen	center_off	all	N	instrument check	14	9	neither
procgen	center_off	all	R	evidence	14	9	neither
procgen	center_off	in_range	I1	evidence	9	4	neither
procgen	center_off	in_range	N	instrument check	9	5	neither
procgen	center_off	in_range	R	evidence	9	5	neither
procgen	center_on	camera_distinct	I1	evidence	5	3	reported, not tested
procgen	center_on	camera_distinct	N	instrument check	5	4	reported, not tested
procgen	center_on	camera_distinct	R	evidence	5	4	reported, not tested

Table 2b. The statistics of the same lines, in the same order.

suite	condition	subset	agent	auroc	ci_low	ci_high	balanced_acc	auroc_B
arc	native	all	N	0.4188	0.1978	0.6379	0.5357	0.5000
arc	native	all	R	0.4100	0.1850	0.6354	0.5333	0.5000
minigrid	macro	primary	I1	0.7187	0.5902	0.8298	0.5838	0.8413
minigrid	macro	primary	N	0.6613	0.5049	0.7941	0.5700	0.8725
minigrid	macro	primary	R	0.6985			0.5693	0.8515
minigrid	macro	primary, D9b included	I1	0.7719	0.6840	0.8529	0.5968	0.8734

suite	condition	subset	agent	auroc	ci_low	ci_high	balanced_acc	auroc_B
minigrid	macro	all games	I1	0.7517	0.6764	0.8223	0.6016	0.7562
minigrid	macro	all games	N	0.6352	0.5514	0.7114	0.5700	0.5752
minigrid	macro	all games	R	0.6434	0.5609	0.7208	0.5693	0.5589
minigrid	raw	primary	I1	0.5370	0.4324	0.6410	0.5070	0.8534
minigrid	raw	primary	N	0.5255	0.4320	0.6156	0.5038	0.6202
minigrid	raw	primary	R	0.5224	0.4330	0.6096	0.5040	0.6561
minigrid	raw	primary, D9b included	I1	0.6258	0.5405	0.7065	0.5070	0.8184
minigrid	raw	all games	I1	0.6069	0.5235	0.6926	0.5070	0.8086
minigrid	raw	all games	N	0.4772	0.3829	0.5696	0.5038	0.6036
minigrid	raw	all games	R	0.4805	0.3919	0.5687	0.5040	0.6472
procgen	center_off	all	I1	0.6458	0.3333	0.9445	0.5000	0.5104
procgen	center_off	all	N	0.7222	0.5000	0.9333	0.5000	0.5667
procgen	center_off	all	R	0.7222	0.5000	0.9333	0.5000	0.5667
procgen	center_off	in_range	I1	0.8750	0.5000	1.0000	0.5000	0.4750
procgen	center_off	in_range	N	0.8000	0.5000	1.0000	0.5000	0.7000
procgen	center_off	in_range	R	0.8000	0.5000	1.0000	0.5000	0.7000
procgen	center_on	camera_distinct	I1	0.3333	0.0000	1.0000	0.5000	0.6667
procgen	center_on	camera_distinct	N	0.3750	0.0000	0.8333	0.5000	1.0000

suite	condition	subset	agent	auroc	ci_low	ci_high	balanced_acc	auroc_B
procgen	center_on	camera_distinct	R	0.3750	0.0000	0.8333	0.5000	1.0000

The analysis code also writes one verdict line per suite, shown in Table 3. Under absolute moves MiniGrid/BabyAI is “not failed”, a verdict decided by I1 alone because R is void. Procgen with the whole-level view is “not failed” over all games and over the in-range games. The primitive-action line reads “suite FAIL”, but the registered text only reports that condition, so this line enters no outcome clause (see Deviations).

Table 3. The V2 suite lines, verbatim from docs/tables/v2.json (commit bb0f1a1).

suite	condition	subset	suite line
arc	native	all	reported, not tested
minigrid	macro	primary	not failed (decided by I1 alone)
minigrid	macro	primary, D9b included	reported, not tested
minigrid	macro	all games	reported, not tested
minigrid	raw	primary	suite FAIL (decided by R and I1)
minigrid	raw	primary, D9b included	reported, not tested
minigrid	raw	all games	reported, not tested
procgen	center_off	all	not failed (decided by R and I1)
procgen	center_off	in_range	not failed (decided by R and I1)
procgen	center_on	camera_distinct	reported, not tested

The nuisance baseline ranked the games better than the census on all six primary MiniGrid/BabyAI lines: the three under absolute moves, which the registered text tests, and the three under primitive actions, which it reports (Figure 2). Over all 14 MiniGrid/BabyAI lines, primary or not, the baseline is higher on 12. The two exceptions are reported lines under absolute moves with all games included, for N (census 0.6352, baseline 0.5752) and for R (census 0.6434, baseline 0.5589). On Procgen the baseline is below the census on every line of the whole-level view.

The two ARC-AGI-3 lines are reported and not tested, as registered. Because of the defect recorded among the deviations, they are computed over all 25 public games rather than the 17 held-out games. The “FAIL” printed on them has no registered meaning and enters no gate. The line over the 17 held-out games is given, as exploratory, in the next section.

Gate G3 applies the registered outcome table (Table 4). The outcome is a refutation, by the clause “V1 fails on any suite”: V1 fails on MiniGrid/BabyAI in both conditions and on Procgen. The other two clauses of the refutation do not apply, since the V2 suite line under absolute moves is “not failed” and V2 is not void on every suite. One clause is enough. This

classification was recorded on 13 October 2026, before any text about the results was written.

Table 4. The registered outcome table, as fixed in the protocol before any confirmatory run (B: MiniGrid/BabyAI; C: Procgen).

outcome	definition (registered)	venue (registered)
Support	V1 passes on B (both conditions) and C, V2 passes for R and I1 on B-macro and C (not void), and V3 holds	JAIR full article
Partial	V1 passes everywhere, but V2 passes on B-macro only or is void on C, or passes on C for R only	IEEE Transactions on Games full paper
Refute	V1 fails on any suite, or V2 fails on B-macro, or V2 is void on every suite	repository record only, at most an arXiv note

Exploratory diagnosis

This section is exploratory. It was written after the outcome was known, it tests nothing, and it changes nothing in the registered results above. Its numbers come from an exploratory script that calls the frozen analysis functions unchanged; the script and its output are named in the data and code availability section. Two consistency checks come first, and both are exploratory. The census verdict stored in each of the 3,500 planner rows equals the verdict recomputed from that row's report, with 0 mismatches. The 25-game ARC-AGI-3 lines recomputed by the same functions equal those of Table 2 to four decimals.

Figure 3 shows, for every V1 line, how the games split between truth and census, and Figure 4 shows how often single seeds disagree with their game's majority. Both are exploratory.

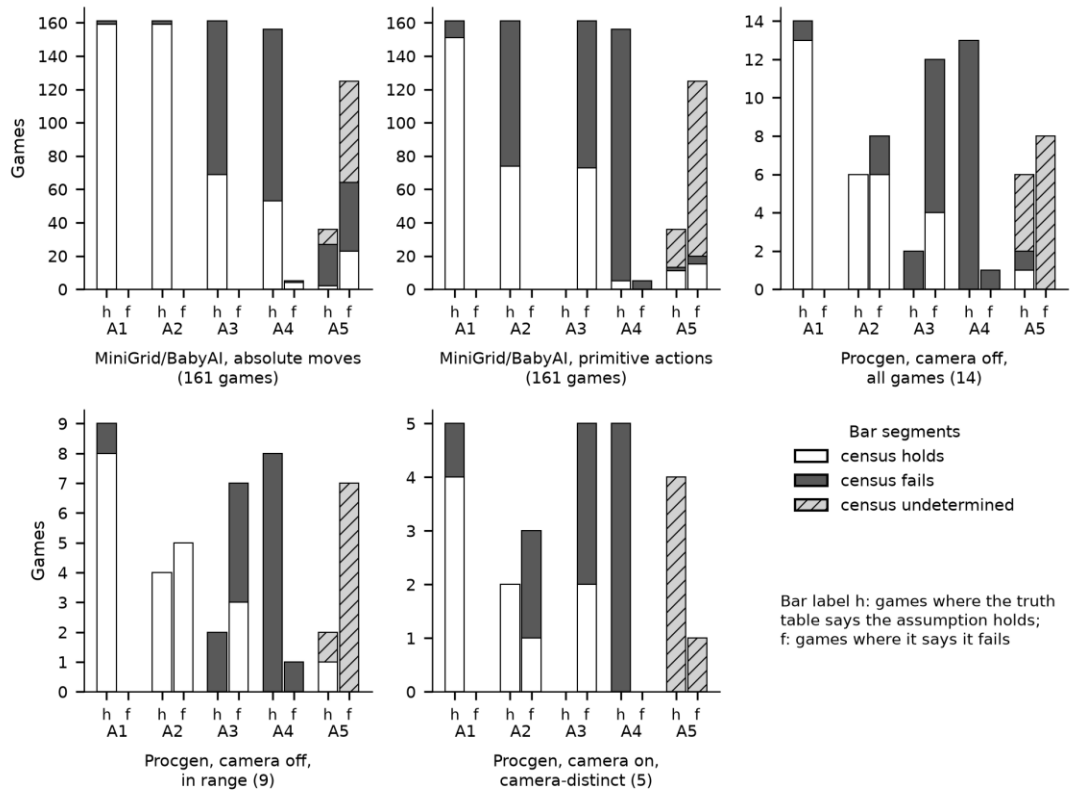


Figure 3. An exploratory view of V1: for each suite and condition and each assumption, the games where the truth table says the assumption holds (bar h) or fails (bar f), split by the census verdict (holds, fails, undetermined). A missing bar is a truth class with zero games. Drawn from runs/exploratory/v1_diagnosis.json by scripts/manuscript_figures.py. Exploratory, read after the outcome was known.

In this exploratory reading, for A1 the census agrees with the truth on 151 of 161 games under primitive actions, 159 of 161 under absolute moves, 13 of 14 on Procgen and 4 of 5 on the camera-on games. Every disagreement is a game where the truth holds and the census fails. On those games most seeds failed the per-seed A1 rule, which fails when no key has a learned shift or when every tried key has the same shift. A1, input coupling, is the assumption that the census and the truth table describe in nearly the same terms, and it is the only one with no failing V1 line.

The exploratory counts for A2 under primitive actions are these: 87 games have the truth holding and the census failing. On their 870 seeds the two turn keys were recorded as producing no movement on all 870, and forward was accepted on 208. A turn changes the agent's facing but not its cell, and the adapter merges the four facings into one tile index, so a turn changes nothing the census can see. Under primitive actions the census's A2 therefore rested on forward alone. Under absolute moves, where every move key displaces the agent, A2 agrees on 159 of 161 games. On Procgen there are 6 games where the truth fails A2 because other entities also respond to the action. The census rule has no clause for that, and on these games it found one controlled object on at least 9 of 10 seeds.

The exploratory counts for A3 under absolute moves are these: 92 games have the truth holding and the census failing. Four step vectors were accepted on 850 of their 920 seeds, so the learned motion was mostly right. Of the 775 seeds whose verdict is “fails”, 755 carry the planner’s first-level outcome “mispredicted” and the other 20 never located the agent. The census’s A3 failure therefore comes from the planner’s prediction during play, not from the learned step vectors. Under primitive actions the truth fails everywhere, because forward depends on facing, and the census says “holds” on 73 games. One step vector per key, learned from the starting facing, cannot reveal a dependence on facing while the turns record no shift.

For A4 the exploratory reading is short. The census rule for A4 is conditional on A3, so most A4 failures on MiniGrid/BabyAI follow from those of A3. On Procgen the counter of walked-through “walls” is above zero on some seeds of all 13 games where the truth holds A4, and on all 10 seeds of 5 of them. That counter rises when the planner walks through a color it had classified as a wall. It counts a color that blocked at one time and not at another, as the planner saw it. The counters do not say whether that came from the game’s obstacles, from momentum or from the planner’s classification.

For A5, the exploratory counts show that most seeds with an undetermined verdict never recorded a goal place at all: 620 of 663 under absolute moves, 986 of 1,074 under primitive actions and 70 of 110 on Procgen. A place is recorded only at a level end, so on these seeds the census had nothing to test. The census said “holds” where the truth fails on 23 games under absolute moves and 15 under primitive actions, and on 0 Procgen games. Success in each of those environments needs facing the target, a toggle or a pickup. They belong to the GoTo, GoToObj, OpenDoor, OneRoom, Fetch and Unlock families. The level then ends on a cell beside or in front of the target, and the census, reading places only, recorded that cell as a goal.

In the same exploratory counts, the negative κ values for A5 come mostly from “undetermined”. The registered κ maps an undetermined verdict to the value opposite the truth. The cell “truth fails, census holds” used by κ therefore contains 23 census “holds” and 61 undetermined games under absolute moves (84 in all). It contains 15 and 105 under primitive actions (120 in all), and 0 and 8 on Procgen (8 in all). On Procgen the negative κ comes entirely from undetermined games.

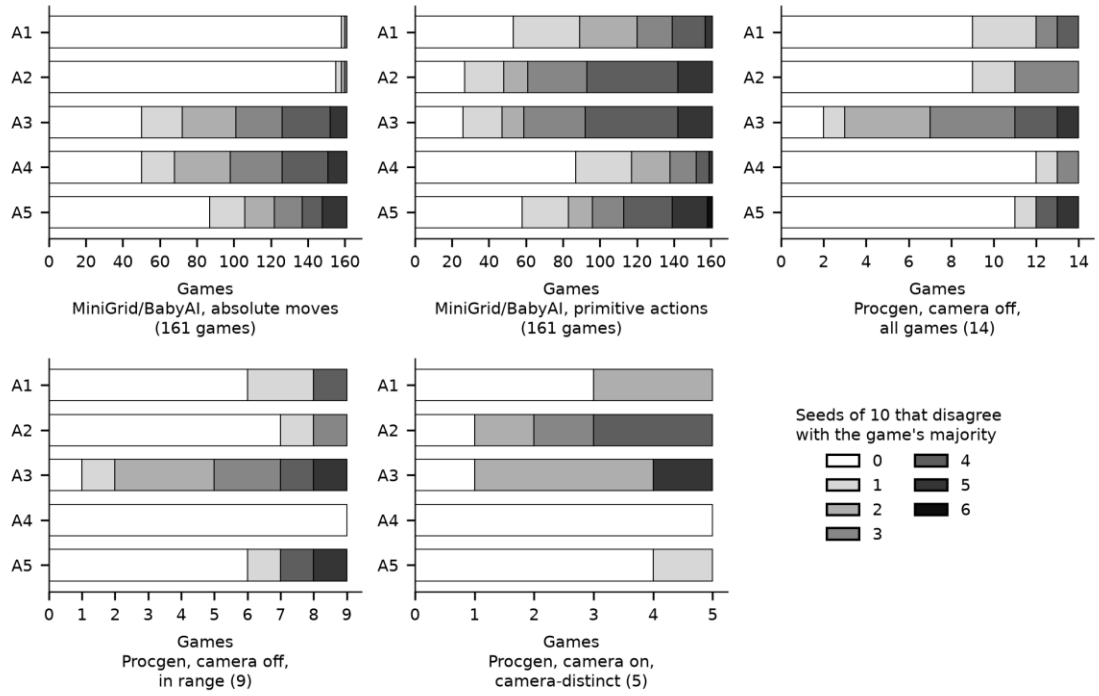


Figure 4. An exploratory view of seed dissent: for each suite and condition and each assumption, the games split by how many of their 10 seeds give a census verdict different from the game's majority verdict. Drawn from runs/exploratory/v1_diagnosis.json by scripts/manuscript_figures.py. Exploratory, read after the outcome was known.

In the same exploratory data, seeds often disagree with their game's majority where the verdict depends on the course of the planner's run: its learning phase for A2, or its first-level outcome for A3 and A4 (Figure 4). At least one seed dissents on 134 of 161 games for A2 under primitive actions, 135 for A3 under primitive actions, and 111 each for A3 and A4 under absolute moves. Dissent is rare for A1 and A2 under absolute moves, on 3 and 6 games, and for A4 on Procgen, on 2 of 14 games.

The exploratory reading that ties these counts together concerns constructs. For A2 under primitive actions, and for A3, A4 and A5, the census probe and the truth definition are not the same construct. The census A2 asks whether some key moves one located object. The truth asks whether exactly one object's displacement is a fixed function of the action, turns included. The census A3 asks whether the planner's first-level model mispredicted. The truth asks whether each action's unblocked displacement is constant. The census A4 is conditional on A3 and counts walked-through "wall" colors. The truth asks whether every blocked move is explained by a fixed obstacle set. The census A5 asks whether a place recorded at a level end ended a level again, and it is silent when no level ends. The truth asks whether success is triggered by entering a place. In each case the census reads a by-product of one planner's run, while the truth reads the environment's step function. This is a description of the mismatch, made after the outcome. It is not a fix and not a result.

Table 5 gives the exploratory ARC-AGI-3 line on the 17 held-out games, computed by the same frozen functions. It is reported and never tested. ARC-AGI-3 rows carry no adapter fields, so the baseline's features are constant there and its AUROC is 0.5000 by construction.

Table 5. Exploratory V2 line on the 17 held-out ARC-AGI-3 games, from runs/exploratory/v1_diagnosis.json. Reported, never tested.

agent	n	positives	AUROC	95% interval	balanced accuracy	baseline AUROC
N	17	8	0.3542	[0.1041, 0.6346]	0.5000	0.5000
R	17	9	0.3403	[0.0909, 0.6213]	0.5000	0.5000

None of this exploratory material supports a causal claim. These are counts of counters, read after the outcome was known.

Purpose

The original objective was to register, and then test without changing it, a method for describing unseen games before any agent plays them. The method was a cheap set of frame-only probes, fitted once, that would report the mechanics an assumption-driven agent depends on and would forecast where such agents finish the first level. Success would have supported a method article; the registered outcome table fixed what each result would permit. The initial hypothesis, as registered on the Open Science Framework on 27 September 2026, reads:

> A fixed, cheap, frame-only probe battery (the assumption census: A1 input coupling, A2 one controlled object, A3 lattice motion, A4 blocking by a fixed obstacle set, A5 place goal), with every parameter frozen on eight ARC-AGI-3 training games, (i) reports the ground-truth mechanics of unseen games at a registered accuracy, and (ii) predicts, before any agent is run, on which unseen games assumption-driven frame-only agents complete the first level. Ground truth is derived from each game's source code and committed before any run. A goal-identification cost G is measured and reported alongside, not tested. ARC-AGI-3 is the development instance and does not count toward generality.

Discussion

What the failure of V1 means, and what it does not

V1 compares the census verdict on each game with that game's entry in the source-derived truth table, and the two do not stand on equal footing. The truth table is the

registered answer key: it was committed before registration, and each entry names the source line that justifies it. The census verdict is the instrument under test. A failing V1 line therefore says that the instrument disagreed with the answer key too often, on the registered games and by the registered thresholds. In plain terms, the census as registered does not measure that assumption on that suite. It says nothing about whether the assumption is a good description of the games, and nothing about whether some other instrument could measure it.

The second half of the claim, prediction, did not pass for any agent either. By the registered outcome table, a V1 failure on any suite decides the outcome by itself, whatever V2 shows. A prediction built on a score that does not measure what it names would also be hard to interpret even if it had passed.

Why the failure could not show on ARC-AGI-3

I set the census constants while developing the planner on eight ARC-AGI-3 games, and I wrote the per-seed rules on top of the planner's report. The protocol provided for a truth table derived from source for those games and for V1 on them, labeled development. I never wrote that table, so V1 was never computed on the development instance. The census was tuned to make the planner work, and nothing on ARC-AGI-3 measured whether its verdicts matched the games' mechanics. A disagreement between census and truth could not show there, because nothing there was looking for it.

What a V1 check on the pilot games would have shown

The pilot runs contained census verdicts for eight games, six MiniGrid/BabyAI environments and two Procgen games, all of which have truth-table entries. Comparing those verdicts with the truth before the v1.1 tag would have shown the disagreement before the 10,500 confirmatory runs were made. The protocol did not register such a check, and I did not make one. The pilot rules allowed adapter code to change after the pilot, but no census constant, threshold or truth entry. An early warning could therefore not have rescued the registered claim. It would have changed what was run next, and how much machine time was spent finding out.

The nuisance baseline

On all six primary MiniGrid/BabyAI lines, a logistic regression on grid area, action count and color count ranked the games by first-level completion better than the census score did. Three of those lines are tested by the registered text and three are reported. How large and how varied an environment is predicted completion better than the five assumptions as measured. This holds for the agents and conditions examined, and it is a statement about the measured census, not about the assumptions as properties of the games. On Procgen the baseline did not rank the games better than the census on any line.

The methodological mistake

The mistake, stated plainly and read from the exploratory diagnosis, is that the probes and the truth predicates were not the same constructs, and that no test existed where the parameters were set. The census decisions were by-products of one planner's run: whether it located an avatar from its first presses, whether its first-level model mispredicted, whether it walked through a color it had called a wall, whether a recorded place ended a level again. The truth entries are properties of the environment's step function. Only A1 is measured in nearly the same terms on both sides, and only A1 had no failing V1 line. I wrote the truth definitions and the census rules separately and never checked on any game, before registration, that one estimated the other.

What a second, separately registered study would have to change

What follows are requirements for a different study, not a result and not an existing instrument. It would have to define each probe and its truth predicate as the same construct before registration, and show on development games that the two agree. It would have to write a truth table for the development instance and compute V1 there before registration, rather than leave it for later as this protocol did. It would have to register a V1 gate on the pilot games, with a stop rule, before any confirmatory run. It would have to register how "undetermined" enters each test, rather than mapping it to the opposite of the truth for κ, so that a silent instrument and a wrong one are told apart. And it would have to name, before registration, a nuisance baseline that the census must beat on the development instance.

What this study does not show

This study does not show that frame-only agents cannot complete these games, and it says nothing about their ability in general. It does not show that A1 to A5 are poor descriptions of the games; the truth tables say how often each holds, and that is not in question. It says nothing about the hidden ARC-AGI-3 games, which were not used, and nothing about agents that bring priors of any kind, visual or learned, to the games. It does not show that a differently built census would fail.

Related work

Bontrager et al. (2016) are the closest prior attempt to predict agent success from game mechanics. Their features were exact by construction, read from the games' rule files, and agreed poorly with controller performance, although a selection agent built on two of them beat its constituent controllers on the same games. The census measured its features from frames and failed at the step before prediction, the measurement itself. Two other lines of work predict success from different information. Laidlaw et al. (2023) predict the sample complexity of deep reinforcement learning from a property of the full tabular decision process with its reward, which a frame-only agent does not have. Gaina et

al. (2018) predict wins on held-out games from an agent's own experience during play, not before it.

The idea behind A1 and A2, finding from frames alone the part of the screen the actions control, comes from Bellemare et al. (2012) and Choi et al. (2019). A2 added a fixed, weight-free test that the controlled region moves by a consistent displacement per key. Under primitive MiniGrid actions that test depended on forward alone, as the exploratory diagnosis shows.

The registered goal-identification cost G counted level ends until one goal hypothesis was left. Its elimination steps over places are the singletons-plus-empty example of Goldman and Kearns (1995), which they state in complement form, and its elimination over features is version-space elimination in the sense of Mitchell (1982). The protocol therefore cited them rather than proving anything new. Goal recognition design is related but different. Keren et al. (2014) bound the number of actions an optimal actor can take before an outside observer can tell its goal, and Wayllace et al. (2016) extend that measure to an expected cost under stochastic action outcomes. G instead concerns a learner's own goal. G was withdrawn without being measured.

Among agents for ARC-AGI-3, Rudakov et al. (2025) published the graph-based explorer used here as I1. Liew (2026) classifies the public games by the simplest strategy without reasoning that reaches a level. That classification concerns reachability rather than mechanics and makes no registered prediction. Skoutnev et al. (2026) identify goals with a coding agent built on a large pretrained model, which writes an executable world model of each game and so brings a learned prior; that places it outside the prior-free class for which G was defined. The two further suites are those of Chevalier-Boisvert et al. (2023), Chevalier-Boisvert et al. (2019) and Cobbe et al. (2020).

Conclusion

The Purpose section states the objective: a frame-only census, fitted on ARC-AGI-3, that would report the mechanics of unseen games at a registered accuracy and predict where assumption-driven agents complete the first level. The research failed against that objective in each of its registered respects. The instrument failed V1 on both confirmatory suites, for three or four of the five assumptions depending on the suite and condition. The prediction test V2 passed for no agent on any suite, and on MiniGrid/BabyAI a baseline built from the size and variety of the environments ranked them better than the census did. The registered consequence followed: a refutation, with no method article. What remains is the record of a test that worked as designed and an instrument that did not.

Data and code availability

All data and code are public in the repository https://github.com/rahul-k-mishra/arc3-frame-only-study under the MIT license, which the journal lists as an acceptable open

license for data and recommends for code. The data are the 10,500 confirmatory rows, 9,660 for MiniGrid/BabyAI and 840 for Procgen, one JSON file per suite, condition, agent, game and seed. They also include the 500-row ARC-AGI-3 baseline, the pilot rows and the outputs of the fidelity, pilot, camera and determinism checks, together with the analysis logs and the V1 and V2 tables. The code is every script that produced them, including the frozen census, the runner, the adapters, the analysis scripts and the three scripts that build and check this article's numbers and figures. The exploratory diagnosis is runs/exploratory/v1_diagnosis.json, written by scripts/explore_v1_diagnosis.py at commit f43526a, and the ARC-AGI-3 split, with the move of dc22, is in docs/HELDOUT.md. The registered protocol is at three tags: prereg-v1 (commit 19f2a9f25043997d1deffd5976f2f2150e5f086f), prereg-v1.1 (commit 8b2b2b9b432dbea4ba5f62cfceb1aad5558d1ecc) and prereg-v1.2 (commit 66e560645d9d98bca39ee7da9b1039ada1bc579b). The two registrations are https://osf.io/2znhw (v1) and https://osf.io/zu5vq (v1.1). Both are under embargo until 30 June 2030, and their registered texts are in the repository. The directory external/rudakov_just_explore, which holds the agent I1, is not covered by the repository's license and keeps the upstream MIT notice of its own authors.

This is the preprint version of an article submitted to the Journal of Trial and Error; it is published under the CC-BY 4.0 license.

Funding

This research received no funding.

Conflicts of interest

The author declares no conflicts of interest.

References

Bellemare, M. G., Veness, J., & Bowling, M. (2012). Investigating contingency awareness using Atari 2600 games. *Proceedings of the AAAI Conference on Artificial Intelligence*, *26*(1), 864–871. https://doi.org/10.1609/aaai.v26i1.8321

Bertinetto, L., Henriques, J. F., Albanie, S., Paganini, M., & Varol, G. (2021). Preface. In L. Bertinetto, J. F. Henriques, S. Albanie, M. Paganini, & G. Varol (Eds.), *NeurIPS 2020 Workshop on Pre-registration in Machine Learning* (Proceedings of Machine Learning Research, Vol. 148, p. i). PMLR. https://proceedings.mlr.press/v148/bertinetto21a.html

Blum, A., & Hardt, M. (2015). The Ladder: A reliable leaderboard for machine learning competitions. In F. Bach & D. Blei (Eds.), *Proceedings of the 32nd International Conference*

on Machine Learning (Proceedings of Machine Learning Research, Vol. 37, pp. 1006–1014). PMLR. https://proceedings.mlr.press/v37/blum15.html

Bontrager, P., Khalifa, A., Mendes, A., & Togelius, J. (2016). Matching games and algorithms for general video game playing. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, *12*(1), 122–128. https://doi.org/10.1609/aiide.v12i1.12884

Chevalier-Boisvert, M., Bahdanau, D., Lahlou, S., Willems, L., Saharia, C., Nguyen, T. H., & Bengio, Y. (2019). BabyAI: A platform to study the sample efficiency of grounded language learning. *International Conference on Learning Representations*. https://openreview.net/forum?id=rJeXCo0cYX

Chevalier-Boisvert, M., Dai, B., Towers, M., Perez-Vicente, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., & Terry, J. (2023). Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems* (Vol. 36, pp. 73383–73394). https://doi.org/10.52202/075280-3209

Choi, J., Guo, Y., Moczulski, M., Oh, J., Wu, N., Norouzi, M., & Lee, H. (2019). Contingency-aware exploration in reinforcement learning. *International Conference on Learning Representations*. https://openreview.net/forum?id=HyxGB2AcY7

Cobbe, K., Hesse, C., Hilton, J., & Schulman, J. (2020). Leveraging procedural generation to benchmark reinforcement learning. In H. Daumé III & A. Singh (Eds.), *Proceedings of the 37th International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 119, pp. 2048–2056). PMLR. https://proceedings.mlr.press/v119/cobbe20a.html

Gaina, R. D., Lucas, S. M., & Perez-Liebana, D. (2018). General win prediction from agent experience. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)* (pp. 1–8). https://doi.org/10.1109/CIG.2018.8490439

Goldman, S. A., & Kearns, M. J. (1995). On the complexity of teaching. *Journal of Computer and System Sciences*, *50*(1), 20–31. https://doi.org/10.1006/jcss.1995.1003

Keren, S., Gal, A., & Karpas, E. (2014). Goal recognition design. *Proceedings of the International Conference on Automated Planning and Scheduling*, *24*(1), 154–162. https://doi.org/10.1609/icaps.v24i1.13617

Laidlaw, C., Russell, S. J., & Dragan, A. (2023). Bridging RL theory and practice with the effective horizon. In *Advances in Neural Information Processing Systems* (Vol. 36, pp. 58953–59007). https://doi.org/10.52202/075280-2573

Liew, K. H. (2026). *Explore before you solve: The speed–depth trade-off in epistemic agents for ARC-AGI-3* [Preprint]. arXiv. https://arxiv.org/abs/2605.25931

Mitchell, T. M. (1982). Generalization as search. *Artificial Intelligence*, *18*(2), 203–226. https://doi.org/10.1016/0004-3702(82)90040-6

Roelofs, R., Shankar, V., Recht, B., Fridovich-Keil, S., Hardt, M., Miller, J., & Schmidt, L. (2019). A meta-analysis of overfitting in machine learning. In *Advances in Neural Information Processing Systems* (Vol. 32). https://papers.nips.cc/paper_files/paper/2019/hash/ee39e503b6bedf0c98c388b7e8589aca-Abstract.html

Rudakov, E., Shock, J., & Cowley, B. U. (2025). *Graph-based exploration for ARC-AGI-3 interactive reasoning tasks* [Preprint]. arXiv. https://arxiv.org/abs/2512.24156

Skoutnev, A., Acharya, K., Longhitano, G., Udell, M., Ellis, K., & Drori, I. (2026). *Twin: Playing an unknown game with a test-time digital twin* [Preprint]. arXiv. https://arxiv.org/abs/2608.14490

Wayllace, C., Hou, P., Yeoh, W., & Son, T. C. (2016). Goal recognition design with stochastic agent action outcomes. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-16)* (pp. 3279–3285). https://www.ijcai.org/Proceedings/16/Papers/464.pdf

Appendix A. The amendment trail, D1 to D16

Each entry is condensed from the diff sections of the registered protocol, where the full text, the file and line references and the dates are given. D1 to D15 were written on 27 September 2026 and registered at prereg-v1.1; D16 was written on 13 October 2026 and registered at prereg-v1.2.

D1, six actions at most. The frozen explorer presses key identifiers 1, 2, 3, 4, 5 and 7 only, so each suite is presented through a fixed key map: turn left, turn right, forward, pickup, drop and toggle under primitive MiniGrid/BabyAI actions; north, south, west, east, pickup and toggle under absolute moves; and up, down, left, right and two special actions on Procgen. An environment with fewer actions keeps only those it has.

D2, canvas placement. A MiniGrid/BabyAI grid sits one row down at the top left of a 64 × 64 canvas, and every other cell holds a reserved padding index, because the frozen code reads rows 1 to 62 only and anchors the frame at the top left.

D3, the Procgen border rows. The 64 × 64 Procgen frame fills the canvas, so the frozen code does not see its top and bottom pixel rows. This is recorded and not corrected.

D4, what a reset does. On MiniGrid/BabyAI a reset replays the current layout, and the layout advances only on a success. Procgen cannot restart a level, so a reset there takes no step and returns the current frame, and it still counts as an action.

D5, the wall-clock cap. The cap per suite is three times the longest pilot run, rounded up to a whole minute and computed by a script: 3300 s for MiniGrid/BabyAI and 7500 s for

Procgen. Pilot runs were uncapped. A run cut by the cap is kept and reported as cut; none was cut.

D6, the runner. One registered script runs both suites, calls the author's agents exactly as the ARC-AGI-3 runner does, refuses to run if any frozen file differs from its registered hash, and writes one row per run.

D7, the boundary of I1. I1 runs unchanged behind an adapter that offers it key identifiers 1 to 5 only, shifts palette values that collide with its status-bar marker, and caps the level count it sees. It therefore lacks toggle on MiniGrid/BabyAI and the second special action on Procgen.

D8, infrastructure checks. The fidelity check, the pilot check, the Procgen camera check and the determinism check play random actions or read existing rows. None computes a census, an agent decision or a test statistic.

D9, exclusions for R and N. An environment is excluded from an agent's primary V2 line when, on every seed, success cannot be reached without an action missing from the agent's key map, either as the success action or as a precondition of every successful episode. The environment with no success at all is on every list. The lists hold 6 environments under primitive actions and 49 under absolute moves.

D9b, exclusions for I1. After the pilot showed I1 completing no level on a locked-door environment, the D9 rule was applied to I1's own key map, adding the 57 environments where success needs toggle on every seed. I1's lists hold 63 and 78 environments. Procgen has no exclusion, because no level end needs the second special action.

D10, seeding text. The v1 seeding text for MiniGrid/BabyAI is replaced by D4, and the runner's commands are added.

D11, the analysis code. The blob hashes of the analysis scripts at the tag are recorded; the scripts changed only to implement D9, D9b and D13. The synthetic tests passed, 20 of 20.

D12, the ARC-AGI-3 cap. The registered ARC-AGI-3 baseline had inherited an unregistered clock of 900 s per run, which cut eight runs short. The cap is 3600 s per run, registered at prereg-v1.1. The eight runs were repeated under the v1 tag, and the repeats replaced the cut rows.

D13, camera identity. Ten Procgen games set the camera option themselves at every level start, so the camera-on condition draws the same frames as the camera-off condition on the nine confirmatory games among them. The camera-on condition is reported on the five other games only and never tested; on the nine, the camera-on runs must equal the camera-off runs, which is checked as a matter of determinism.

D14, no adapter change after the pilot. The pilot showed no errors, no cut runs and the expected identity between conditions, and no adapter was changed.

D15, G by a registered replay. The rows for the goal-identification cost G were to come from a separate replay of N, registered as a further version before any replay, with every replay required to reproduce the confirmatory actions exactly.

D16, G is not measured. The replay of D15 is withdrawn after the outcome, because the instrument that G would accompany failed V1 on both suites. G remains defined in the protocol, and no value of it is reported.

Appendix B. The exclusion lists

These lists are the registered constants in scripts/analyze_common.py at prereg-v1.1. The source reason for each identifier is given in the protocol under D9 and D9b.

B1. No success at all (1 environment; on every agent's list in both conditions):

- MiniGrid-Playground-v0

B2. Success needs "done" (5; on every list in both conditions; "done" is in no key map):

- MiniGrid-GoToDoor-5x5-v0
- MiniGrid-GoToDoor-6x6-v0
- MiniGrid-GoToDoor-8x8-v0
- MiniGrid-GoToObject-6x6-N2-v0
- MiniGrid-GoToObject-8x8-N2-v0

B3. Success is a drop (15; on the absolute-move lists of every agent):

- MiniGrid-PutNear-6x6-N2-v0
- MiniGrid-PutNear-8x8-N3-v0
- BabyAI-PutNextLocal-v0
- BabyAI-PutNextLocalS5N3-v0
- BabyAI-PutNextLocalS6N4-v0
- BabyAI-PutNextS4N1-v0
- BabyAI-PutNextS5N1-v0
- BabyAI-PutNextS5N2-v0
- BabyAI-PutNextS5N2Carrying-v0
- BabyAI-PutNextS6N3-v0
- BabyAI-PutNextS6N3Carrying-v0
- BabyAI-PutNextS7N4-v0
- BabyAI-PutNextS7N4Carrying-v0
- BabyAI-MoveTwoAcrossS5N2-v0
- BabyAI-MoveTwoAcrossS8N9-v0

B4. A drop is a precondition (28; on the absolute-move lists of every agent, and on I1's lists in both conditions as toggle preconditions):

- MiniGrid-KeyCorridorS3R1-v0
- MiniGrid-KeyCorridorS3R2-v0
- MiniGrid-KeyCorridorS3R3-v0
- MiniGrid-KeyCorridorS4R3-v0
- MiniGrid-KeyCorridorS5R3-v0
- MiniGrid-KeyCorridorS6R3-v0
- MiniGrid-UnlockPickup-v0
- MiniGrid-BlockedUnlockPickup-v0
- MiniGrid-ObstructedMaze-1Dl-v0
- MiniGrid-ObstructedMaze-1Dlh-v0
- MiniGrid-ObstructedMaze-1Dlhb-v0
- MiniGrid-ObstructedMaze-1Q-v1
- MiniGrid-ObstructedMaze-2Dl-v0
- MiniGrid-ObstructedMaze-2Dlh-v0
- MiniGrid-ObstructedMaze-2Dlhb-v1
- MiniGrid-ObstructedMaze-2Q-v1
- MiniGrid-ObstructedMaze-Full-v1
- BabyAI-KeyCorridor-v0
- BabyAI-KeyCorridorS3R1-v0
- BabyAI-KeyCorridorS3R2-v0
- BabyAI-KeyCorridorS3R3-v0
- BabyAI-KeyCorridorS4R3-v0
- BabyAI-KeyCorridorS5R3-v0
- BabyAI-KeyCorridorS6R3-v0
- BabyAI-UnlockPickup-v0
- BabyAI-UnlockPickupDist-v0
- BabyAI-BlockedUnlockPickup-v0
- BabyAI-UnlockToUnlock-v0

B5. Success is a toggle (20; on I1's lists in both conditions):

- MiniGrid-Unlock-v0
- MiniGrid-RedBlueDoors-6x6-v0
- MiniGrid-RedBlueDoors-8x8-v0
- BabyAI-Open-v0
- BabyAI-OpenDoor-v0

- BabyAI-OpenDoorColor-v0
- BabyAI-OpenDoorDebug-v0
- BabyAI-OpenDoorLoc-v0
- BabyAI-OpenDoorsOrderN2-v0
- BabyAI-OpenDoorsOrderN2Debug-v0
- BabyAI-OpenDoorsOrderN4-v0
- BabyAI-OpenDoorsOrderN4Debug-v0
- BabyAI-OpenRedBlueDoors-v0
- BabyAI-OpenRedBlueDoorsDebug-v0
- BabyAI-OpenRedDoor-v0
- BabyAI-OpenTwoDoors-v0
- BabyAI-Unlock-v0
- BabyAI-UnlockLocal-v0
- BabyAI-UnlockLocalDist-v0
- BabyAI-KeyInBox-v0

B6. A toggle is a precondition (these 9, plus the 28 of B4; on I1's lists in both conditions):

- MiniGrid-DoorKey-5x5-v0
- MiniGrid-DoorKey-6x6-v0
- MiniGrid-DoorKey-16x16-v0
- MiniGrid-LockedRoom-v0
- MiniGrid-MultiRoom-N2-S4-v0
- MiniGrid-MultiRoom-N4-S5-v1
- MiniGrid-MultiRoom-N6-v0
- BabyAI-GoToImpUnlock-v0
- BabyAI-PickupAbove-v0

Table B1. The lists per agent and condition.

agent and condition	lists	excluded	primary games
R and N, primitive actions	B1, B2	6	155
R and N, absolute moves	B1, B2, B3, B4	49	112
I1, primitive actions	B1, B2, B4, B5, B6	63	98
I1, absolute moves	B1, B2, B3, B4, B5, B6	78	83
any agent, Procgen	none	0	14

Appendix C. Provenance of the numbers

Every number in this article is a value in docs/manuscript/numbers.json, written by scripts/manuscript_numbers.py from committed files, and scripts/manuscript_check.py rejects any numeral in the text that is not one of those values. Table C1 lists, for each class of number, the committed files its values come from and the commit that last changed each file. It is written by scripts/manuscript_numbers.py and copied here unchanged.

Table C1. Number classes and their sources.

number class	source files, each with the commit that last changed it
count	docs/EXPLORATORY_V1_DIAGNOSIS.md (7d9205e); docs/GROUND_TRUTH_minigrid.md (f12e529); docs/GROUND_TRUTH_procgen.md (8d209b3); docs/HELDOUT.md (4d2c02c); docs/PREREG.md (7d9205e); docs/STATE.md (2048f36); docs/tables/v1.json (bb0f1a1); docs/tables/v2.json (bb0f1a1); runs/checks/determinism_check.json (5ac65c1); runs/exploratory/v1_diagnosis.json (66e5606); runs/pilot_checks/pilot_check.json (cda7aa0); runs/pilot_checks/procgen_camera_check.json (cda7aa0); scripts/analyze_common.py (a96e49e)
rate (agreement, κ, AUROC, interval bound, share)	docs/tables/v1.json (bb0f1a1); docs/tables/v2.json (bb0f1a1); runs/exploratory/v1_diagnosis.json (66e5606)
registered threshold	docs/PREREG.md (7d9205e)
constant, cap, seed or time	docs/PREREG.md (7d9205e); docs/tables/v2.json (bb0f1a1); runs/pilot_checks/pilot_check.json (cda7aa0); runs/pilot_checks/procgen_camera_check.json (cda7aa0)
commit or blob hash	docs/PREREG.md (7d9205e);

number class	source files, each with the commit that last changed it
	docs/STATE.md (2048f36); git tag objects and git ls-tree (the tag objects themselves)
date	docs/HELDOUT.md (4d2c02c); docs/PREREG.md (7d9205e); docs/STATE.md (2048f36); git tag objects and git ls-tree (the tag objects themselves)
package version	docs/PREREG.md (7d9205e)
URL or DOI	docs/STATE.md (2048f36); docs/manuscript/REFERENCES_CHECK.md (6df6718); docs/manuscript/REVIEW_draft1.md (48931d2); docs/preprint/preprint.md (8bed48b); docs/preprint/references.bib (8bed48b)
reference field, verdict word or ORCID	docs/manuscript/REFERENCES_CHECK.md (6df6718); docs/preprint/preprint.md (8bed48b); docs/preprint/references.bib (8bed48b); docs/tables/v1.json (bb0f1a1); docs/tables/v2.json (bb0f1a1)

Appendix D. Frozen files and analysis scripts

The six census files were frozen at prereg-v1 and are identical at prereg-v1.1; the runner checked their blob hashes before every run. The analysis scripts are those at prereg-v1.1. Each hash below equals the output of git ls-tree at prereg-v1.1.

Table D1. Git blob hashes of the frozen files and the analysis scripts.

file	role	git blob hash
scripts/g_agent.py	explorer R, frozen	72585a4dbb926d4512ff9596f58118a176bb0646
scripts/g_nav.py	planner N and census report, frozen	a8a7eb61e62a658c239b98e0224e3588a662d1a0
scripts/g_motion.py	motion detector, frozen	14018512a5c6b0efc81aed61c5466654d338be0b
scripts/g_control_probe.py	key set, frozen	d5ff2a4dc77b9e1148929e7db03fe99a9d2b0bf8
scripts/g_nav_run.py	ARC-AGI-3 runner, frozen	2cc0146a697e525e5873d4e29cb80f4ae8bd6297

file	role	git blob hash
scripts/g_census_table.py	ARC-AGI-3 census table rule, frozen	488feaca9c10985c4d0ff62791ceb4fa6fa6b9a8
scripts/analyze_common.py	census rules, statistics, exclusion lists	caad22731807db78aff9c92c993a79a63f042ec3
scripts/analyze_v1.py	test V1	515f44b0f007182172022272383de548e80d5a11
scripts/analyze_v2.py	test V2	90a849c44e47addd8efdbfa579efdb16288edd50
scripts/analyze_g.py	cost G (not run)	46bf991d86400f0189d8898a7e068044d25c629a
scripts/determinism_check.py	determinism check	4f98f6430e9221deb1a43f2313eb00f5ca676969